Computer Science Fundamentals

Source Coding – Huffman-Algorithm

Technische Hochschule Rosenheim
Winter 2025/26
Prof. Dr. Jochen Schmidt

Overview

Technische Hochschule Rosenheim

- Code trees
- Huffman algorithm

Code Trees

Technische Hochschule Rosenheim

- Objective: Creation of a code with variable word length
  - for a given alphabet of symbols
  - with known probabilities of occurrence

- Example for a simple procedure
  - Determine the word length from the symbol information contents rounded up to nearest integer
  - Arrange code words as end nodes (leaves) of a code tree
  - Note: This is in general suboptimal and NOT how we'll do it!

Code Trees – Example Simple Code Creation

Technische Hochschule Rosenheim

- 6 letters {c, v, w, u, r, z} are to be binary encoded with as little redundancy as possible
- Probabilities of occurrence

$$
\begin{aligned}
p(c) &= 0.1643 \\
p(v) &= 0.0455 \\
p(w) &= 0.0874 \\
p(u) &= 0.1963 \\
p(r) &= 0.4191 \\
p(z) &= 0.0874
\end{aligned}
$$

Code Trees – Example Simple Code Creation

Technische Hochschule Rosenheim

- Calculation of the information content of the respective letters

$$I(x) = \mathrm{ld}\frac{1}{p(x)} \; [Bit]$$

$p(c)$	=	0.1643	$I(c)$	=	2.6056 Bit
$p(v)$	=	0.0455	$I(v)$	=	4.4580 Bit
$p(w)$	=	0.0874	$I(w)$	=	3.5162 Bit
$p(u)$	=	0.1963	$I(u)$	=	2.3489 Bit
$p(r)$	=	0.4191	$I(r)$	=	1.2546 Bit
$p(z)$	=	0.0874	$I(z)$	=	3.5162 Bit

Code Trees – Example Simple Code Creation

Technische Hochschule Rosenheim

- Calculation of the information content of the respective letters and derivation of the word length

$$
\begin{aligned}
I(c) &= 2.6056\ \text{Bit} \longrightarrow l(c) = 3\ \text{Bit} \\
I(v) &= 4.4580\ \text{Bit} \longrightarrow l(v) = 5\ \text{Bit} \\
I(w) &= 3.5162\ \text{Bit} \longrightarrow l(w) = 4\ \text{Bit} \\
I(u) &= 2.3489\ \text{Bit} \longrightarrow l(u) = 3\ \text{Bit} \\
I(r) &= 1.2546\ \text{Bit} \longrightarrow l(r) = 2\ \text{Bit} \\
I(z) &= 3.5162\ \text{Bit} \longrightarrow l(z) = 4\ \text{Bit}
\end{aligned}
$$

Code Trees – Example Simple Code Creation

- Calculation of the information content of the respective letters and derivation of the word length with encoding

$$
\begin{array}{llllll}
I(c) & = & 2.6056 \text{ Bit} & \longrightarrow & l(c) = 3 \text{ Bit} & \texttt{001} \\
I(v) & = & 4.4580 \text{ Bit} & \longrightarrow & l(v) = 5 \text{ Bit} & \texttt{10111} \\
I(w) & = & 3.5162 \text{ Bit} & \longrightarrow & l(w) = 4 \text{ Bit} & \texttt{0001} \\
I(u) & = & 2.3489 \text{ Bit} & \longrightarrow & l(u) = 3 \text{ Bit} & \texttt{011} \\
I(r) & = & 1.2546 \text{ Bit} & \longrightarrow & l(r) = 2 \text{ Bit} & \texttt{11} \\
I(z) & = & 3.5162 \text{ Bit} & \longrightarrow & l(z) = 4 \text{ Bit} & \texttt{0000}
\end{array}
$$

Exemplary Code

Code Trees – Example Simple Code Creation

Technische Hochschule Rosenheim

- Entropy of this data source

$$\begin{aligned} H &= p(c)I(c) + p(v)I(v) + p(w)I(w) + p(u)I(u) + p(r)I(r) + p(z)I(z) \\ &\approx 0.4282 + 0.2028 + 0.3073 + 0.4611 + 0.5260 + 0.3073 \\ &= 2.2327 \text{ bits/symbol} \end{aligned}$$

- maximum entropy possible with 6 symbols

$$H_0 = \text{ld } 6 = 2.5850$$

Symbol	$p(c)$	$I(c)$	$l(c)$
c	0.1643	2.6056	3
v	0.0455	4.4580	5
w	0.0874	3.5162	4
u	0.1963	2.3489	3
r	0.4191	1.2546	2
z	0.0874	3.5162	4

- Average word length

$$\begin{aligned} L &= p(c)l(c) + p(v)l(v) + p(w)l(w) + p(u)l(u) + p(r)l(r) + p(z)l(z) \\ &= 0.4929 + 0.2275 + 0.3496 + 0.5889 + 0.8382 + 0.3496 \\ &= 2.8467 \text{ bits/symbol} \end{aligned}$$

- Redundancy

Code

$$\begin{aligned} R &= L - H \\ &\approx 2.8467 - 2.2327 \\ &= 0.6140 \text{ bits/symbol} \end{aligned}$$

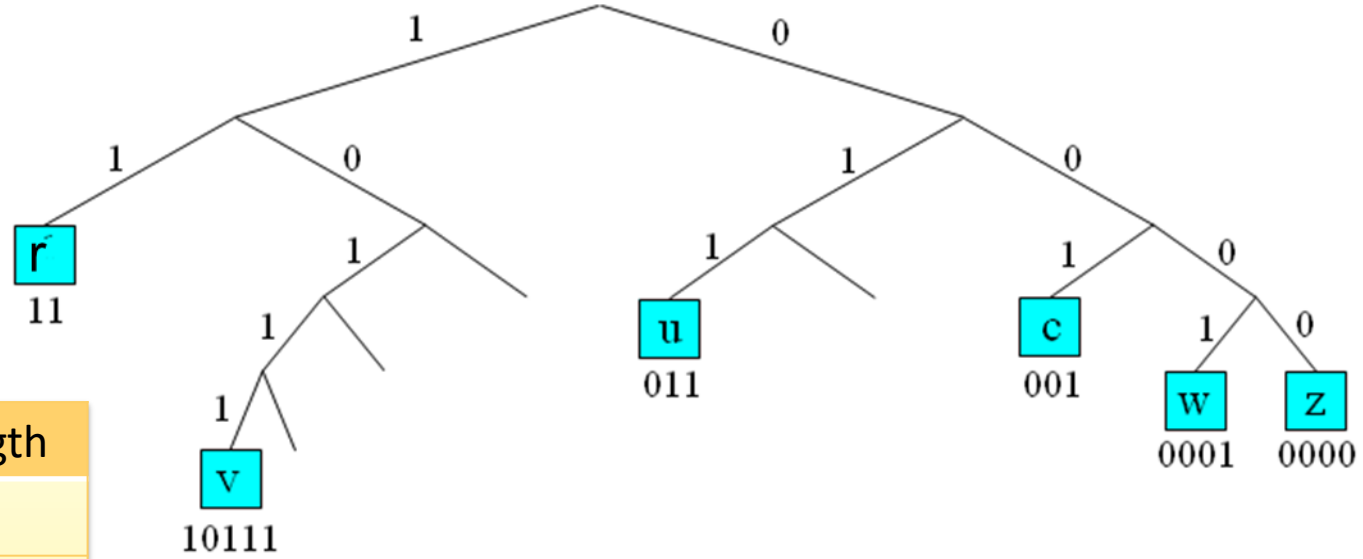
Source

$$\begin{aligned} R_S &= H_0 - H \\ &\approx 2.5850 - 2.2327 \\ &= 0.3523 \text{ bits/symbol} \end{aligned}$$

Code Trees – Example Simple Code Creation

Technische Hochschule Rosenheim

Code visualization



Symbol	Code	Length
c	001	3
v	10111	5
w	0001	4
u	011	3
r	11	2
z	0000	4

Decoding – Prefix Code

Technische Hochschule Rosenheim

- Essential requirement for a code: unique decoding
- block codes: easy, each code word has the same length
- variable-length codes: also easy, if the code is a prefix code
  - no code word is prefix of any other code word, i.e.,
  - code words can only be on the leaves of the code tree
- a better term would actually be prefix-free code
- Huffman automatically generates prefix codes

Decoding Process

Technische Hochschule Rosenheim

1. The code words to be interpreted are collected bit by bit in a buffer and continuously compared with the tabulated code words.
   - Due to the prefix-condition, this is always unique.

2. Once the buffer content matches a tabulated code word, decoding is performed.

3. The buffer is cleared, and the decoding operation starts again for the next code word.

Decoding – Exercise

Technische Hochschule Rosenheim

Decode the message 00101100101111010110000 using the code from before

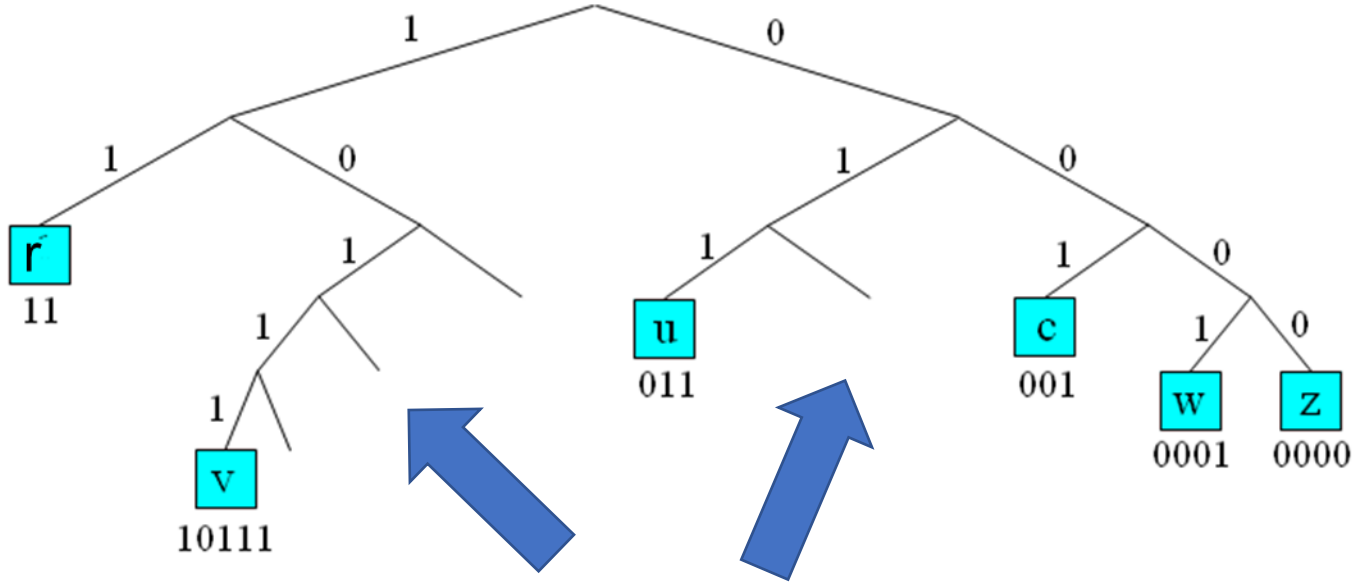
Symbol	Code
c	001
v	10111
w	0001
u	011
r	11
z	0000

Solution:

Code Trees – Example Simple Code Creation

Technische Hochschule Rosenheim

Code visualization



- Unoccupied leaves (end nodes) are present that are closer to the root
- Not an optimal code: shorter code words would be possible

Code Trees – Example Simple Code Creation

Technische Hochschule Rosenheim

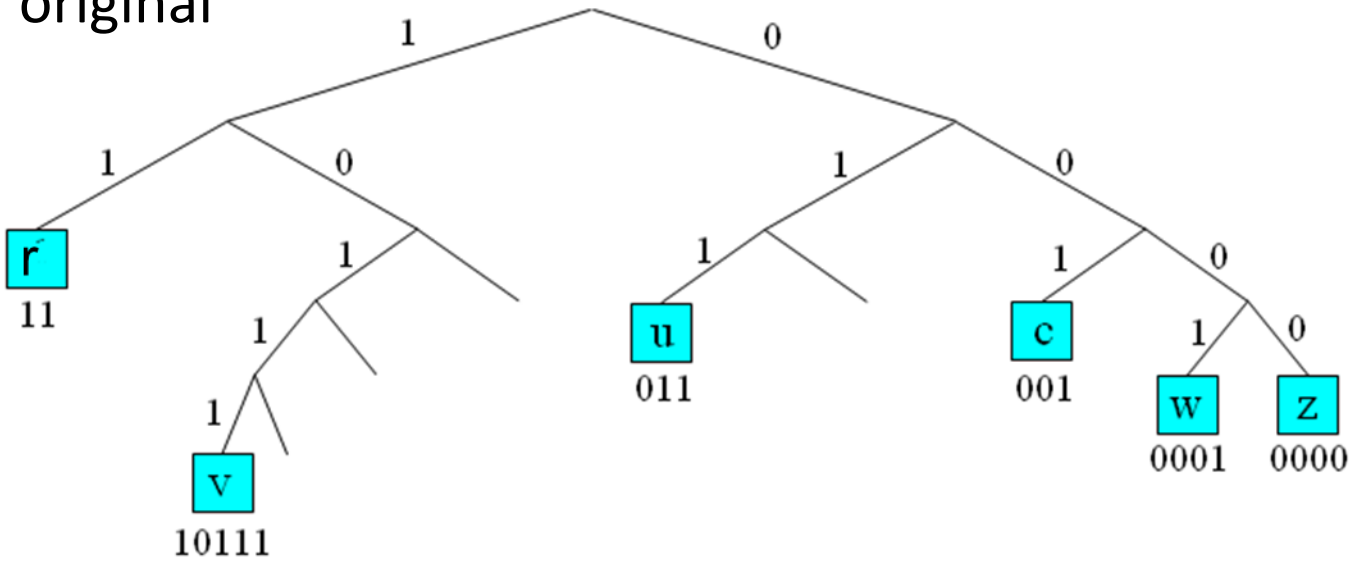
Improvements

- Move symbols such that we get rid of unoccupied leaves
- Caution
  - A code for a symbol with a lower probability of occurrence must be longer than the code for a symbol with a higher probability of occurrence

Code Trees – Example Simple Code Creation

Technische Hochschule Rosenheim

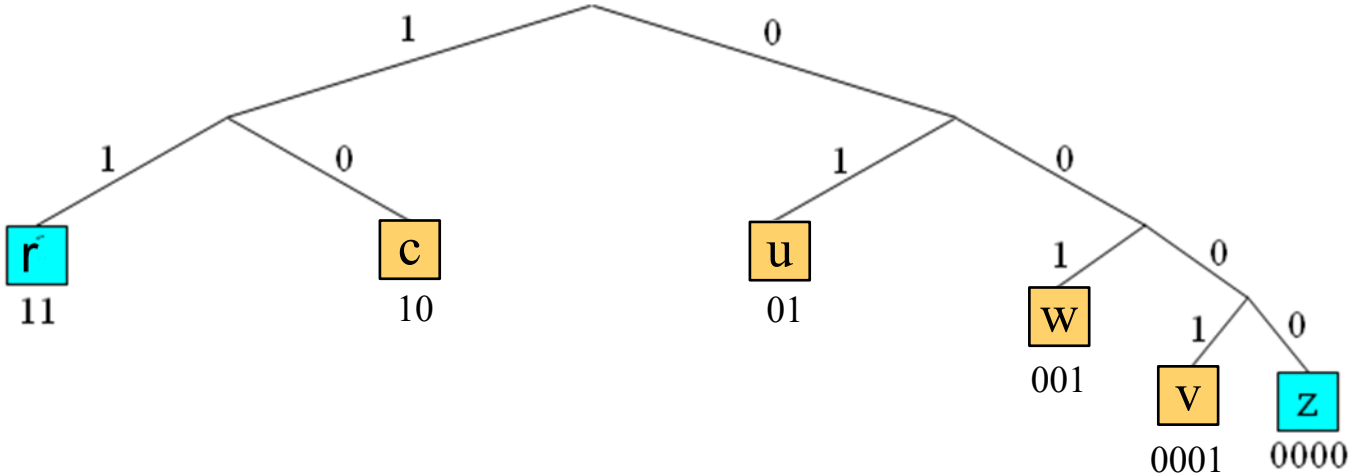
original



Symbol	Code	Length
c	**001**	**3**
v	**10111**	**5**
w	**0001**	**4**
u	**011**	**3**
r	**11**	**2**
z	**0000**	**4**

improved

Symbol	Code	Length
c	**10**	**2**
v	**0001**	**4**
w	**001**	**3**
u	**01**	**2**
r	**11**	**2**
z	**0000**	**4**



Code Trees – Example Simple Code Creation

Technische Hochschule Rosenheim

Results of optimization

Symbol	$p(c)$	$I(c)$	$l(c)$
c	0.1643	2.6056	2
v	0.0455	4.4580	4
w	0.0874	3.5162	3
u	0.1963	2.3489	2
r	0.4191	1.2546	2
z	0.0874	3.5162	4

- Average word length of the improved code

  $L =$

  $= 2.3532$ bits/symbol

- Redundancy

  $R \approx$ $= 0.1205$ bits/symbol

- Redundancy was reduced from 0.6140 bits/symbol to 0.1205 bits/symbol

- However:
  - This was tedious – it has to be done automatically by a computer. How would we implement this?
  - Is this the best we can do? Or can we get an even better code by re-arranging the code words?

Huffman Algorithm

Technische Hochschule Rosenheim

- 1952 by David A. Huffman
- Algorithm for
  - generating optimal codes
  - with regard to the criterion redundancy minimization considering single symbols
- Start with individual symbols and combine them (bottom-up)

Huffman Algorithm – Example

Technische Hochschule Rosenheim

Step-by-step construction of the Huffman tree and Huffman code

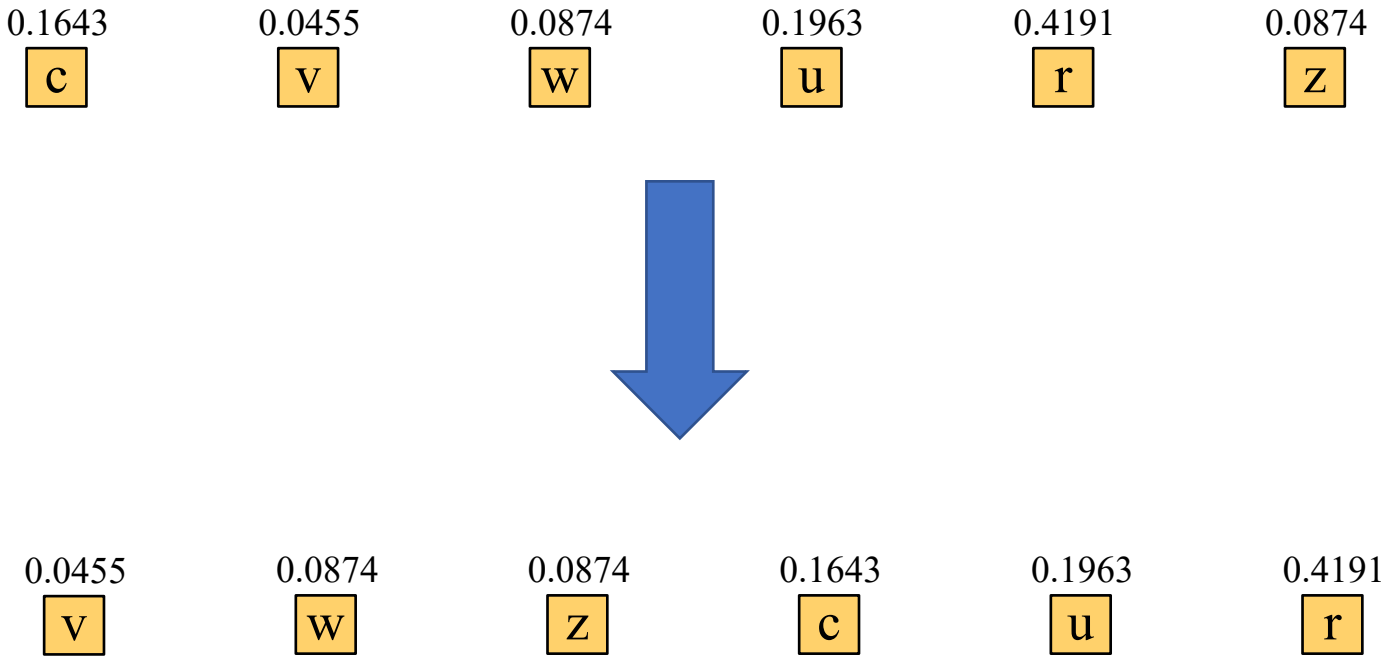
Starting point: 6 letters {c, v, w, u, r, z} with the probabilities of occurrence

$$
\begin{aligned}
p(c) &= 0.1643 \\
p(v) &= 0.0455 \\
p(w) &= 0.0874 \\
p(u) &= 0.1963 \\
p(r) &= 0.4191 \\
p(z) &= 0.0874
\end{aligned}
$$

Huffman Algorithm – Example

Technische Hochschule Rosenheim

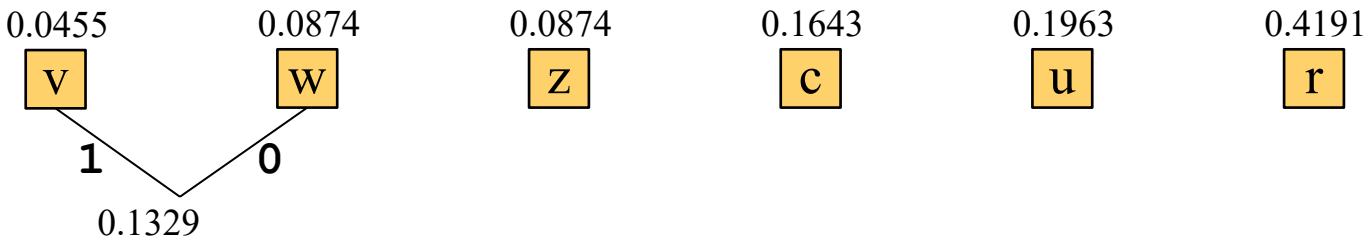
Step 1: Sort symbols by ascending probability



Huffman Algorithm – Example

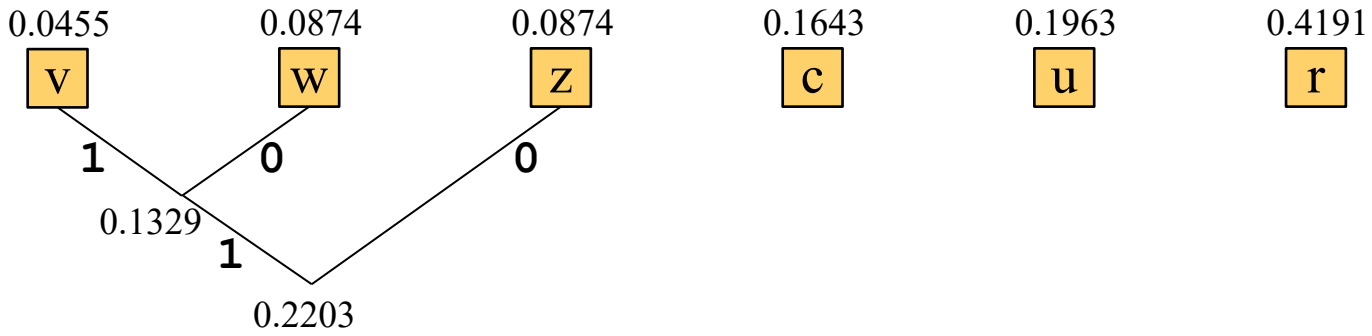
Technische Hochschule Rosenheim

Step 2: Combine the two symbols with smallest probability



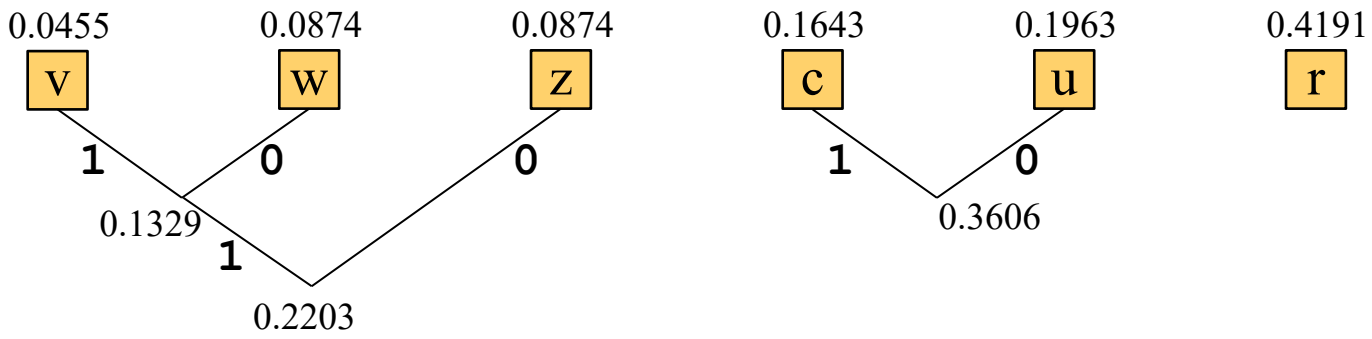
Huffman Algorithm – Example

Step 3: Treat combination as new “symbol”
and again combine the two smallest probabilities (in an algorithm: sort again)



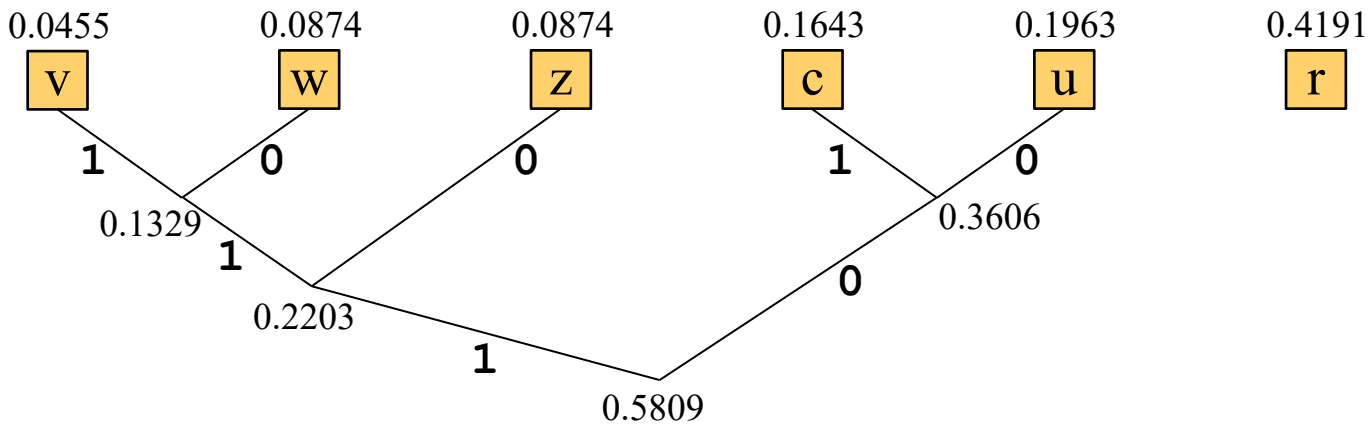
Huffman Algorithm – Example

… and so on…



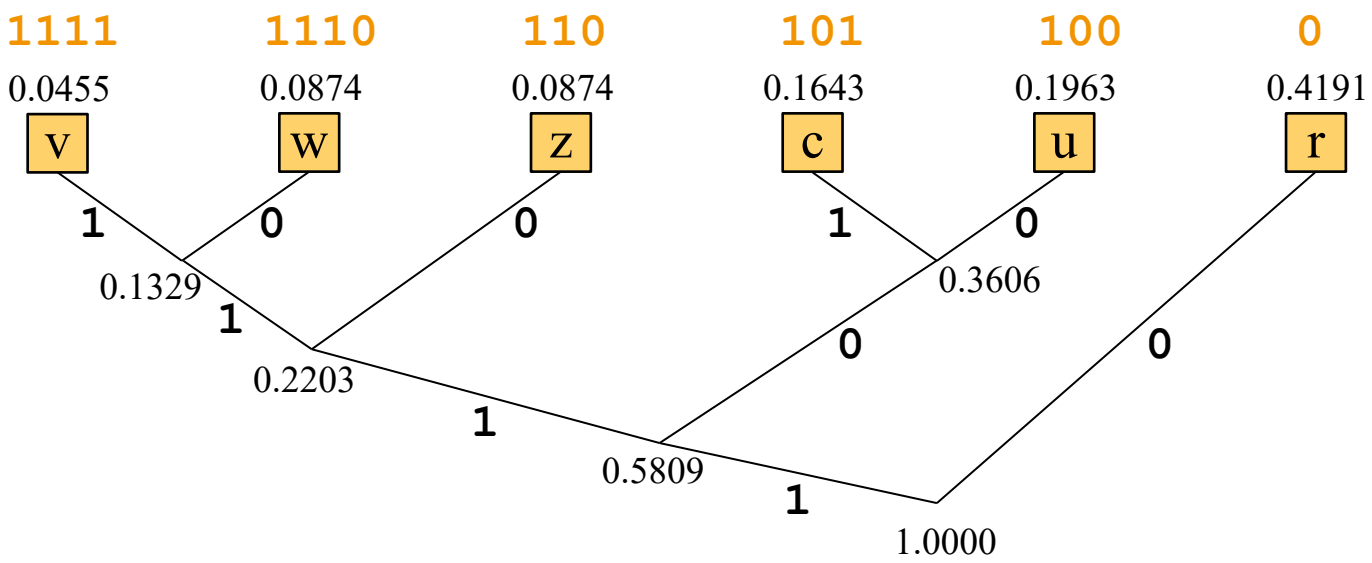
Huffman Algorithm – Example

… and so on…



Huffman Algorithm – Example

… and so on…



Huffman Algorithm – Example

Technische Hochschule Rosenheim

- Average code word length

$$L = 0.4191 + (0.1963 + 0.1643 + 0.0874)\cdot 3 + (0.0874 + 0.0455)\cdot 4 = 2.2947 \text{ bits/symbol}$$

- Redundancy

$$R = 2.2947 - 2.2327 = 0.0620 \text{ bits/symbol}$$

Symbol c	$p(c)$	$I(c)$	$l(c)$
c	0.1643	2.6056	3
v	0.0455	4.4580	4
w	0.0874	3.5162	4
u	0.1963	2.3489	3
r	0.4191	1.2546	1
z	0.0874	3.5162	3

- Comparison to previous codes
  - First, redundancy was reduced from 0.6140 bits/symbol to 0.1205 bits/symbol
  - Now further reduction of redundancy to 0.062 bits/symbol

Huffman Algorithm – Summary

Technische Hochschule Rosenheim

1. Sort all symbols ascending by to their probabilities of occurrence (leaves of the code tree)

2. Combine the two symbols with the lowest probabilities p_1 und p_2 to a node
   - Probability = $p_1 + p_2$
   - Result: New sequence of probabilities

3. Combine the two elements (single symbols and/or nodes) with the lowest probabilities to a new node

4. Repeat (3) until everything has been combined and probability 1 results (result: Huffman Tree)

Summary

Technische Hochschule Rosenheim

Huffman codes are proven to be optimal

- in the sense of the shortest possible average word length for
  - separate encoding of single symbols,
  - an integral number of bits per symbol (i.e., no fractional bits for a symbol).
- and therefore in the sense of obtaining the smallest possible redundancy
- optimal means: **there exists no better coding algorithm in the above sense**!
- they are widely used in compression algorithms
  - often as a final step (so-called entropy coding) after lossy compression
  - often combined with run-length encoding